

ADS Project

Social Media Platform Using Advanced Data Structures

Project Idea

This project is a console-based social media platform where users can register, log in, create posts, follow other users, like and comment on posts, view a personalized feed, receive notifications, view trending posts, and undo recent actions.

Data Structures Shortlist

Data Structure	Main Function in the Platform
Hash Map / Hash Table	Store users and posts for fast access.
Graph	Represent follower/following relationships between users.
Adjacency List	Store each user's follow connections.
Priority Queue / Max Heap	Display trending posts or ranked feed.
Queue	Store and process notifications.
Stack	Undo recent user actions.
Linked List	Store comments under each post.
AVL Tree / Balanced BST	Keep posts or users sorted.

Basic Platform Flow

```
Start Program
-> Register or Login
-> User Dashboard
-> Create posts, follow users, like/comment, view feed, view notifications
-> System updates the correct data structures
-> User logs out or exits
```

Main Menu Example

1. Register
2. Login
3. View Trending Posts
4. Exit

User Dashboard Example

1. Create Post
2. View My Profile
3. Follow User
4. Unfollow User
5. View Feed
6. Like a Post
7. Comment on a Post
8. View Trending Posts
9. View Notifications
10. Undo Last Action
11. Logout

Testing Demo

Sample walkthrough to show colleagues how the data structures work during execution.

Demo Setup

Create three users and a few posts so the system has enough data to test follows, feed ranking, notifications, comments, and undo.

Register users:

- Andrew
- Sara
- Omar

Create posts:

- Sara creates Post #101: "Studying ADS today"
- Omar creates Post #102: "New project idea"
- Andrew creates Post #103: "Hello everyone"

Test Case 1 - Follow System

Action: Andrew follows Sara and Omar.

Expected Graph / Adjacency List	Result
Andrew: Sara, Omar Sara: - Omar: -	Andrew now follows Sara and Omar. The feed can now collect Sara and Omar's posts.

Test Case 2 - Personalized Feed

Action: Andrew chooses "View Feed".

System steps:

1. Check Andrew's following list: Sara, Omar
2. Collect posts from Sara and Omar
3. Insert those posts into the Priority Queue / Max Heap
4. Display the highest-ranked posts first

Expected feed output:

1. Sara - Post #101
2. Omar - Post #102

Test Case 3 - Like, Comment, and Notification Queue

Action: Andrew likes Sara's post and comments on it.

Expected changes:

- Post #101 like count increases by 1
- Andrew's comment is added to Post #101 using the comments Linked List
- Sara receives two notifications in her Queue:
 1. Andrew liked your post
 2. Andrew commented on your post

Test Case 4 - Trending Posts

Action: The system displays trending posts.

Example scores:

- Post #101 score = likes + comments + recency bonus = 20
- Post #102 score = likes + comments + recency bonus = 12
- Post #103 score = likes + comments + recency bonus = 5

Expected trending output:

1. Post #101
2. Post #102
3. Post #103

Test Case 5 - Undo Last Action

Action: Andrew chooses "Undo Last Action".

If Andrew's last action was commenting on Sara's post:

- The system pops the last action from the Stack
- The comment is removed from Post #101
- The notification can be updated or marked as reversed

Expected output:

"Your last comment was removed successfully."

Testing Checklist

Feature	What to Check
Register/Login	Users are saved and retrieved correctly from the Hash Map.
Follow/Unfollow	The adjacency list updates correctly.
Feed	Only followed users' posts appear.
Trending	Posts are displayed from highest to lowest rank.
Notifications	Notifications appear in the order they were received.
Undo	The most recent action is reversed first.
Comments	Comments remain attached to the correct post.

Final Demo Goal

By the end of the demo, the audience should clearly see that each feature is connected to a specific data structure: Hash Map for storage, Graph/Adjacency List for follows, Priority Queue for feed and trending posts, Queue for notifications, Stack for undo, Linked List for comments, and AVL Tree/Balanced BST for sorted views.